Cloud Computing — Infrastructure Project

Mini Distributed Cloud Object Store

1. Project Objective

Build a distributed cloud object storage system capable of storing and retrieving arbitrary objects across multiple storage nodes. The system should support documents, images, videos, audio files, compressed files, and other binary data. Objects must be divided into chunks and stored across multiple workers with replication.

2. System Configuration

Parameter	Requirement
Controller nodes	1
Storage/Worker nodes	4
Clients	1 or more
Chunk size	1 MB
Replication factor	2
Deployment	Docker
Worker storage	Persistent storage

The complete system must be runnable locally using Docker. Docker Compose may be used. Kubernetes should not be used for worker orchestration.

3. Object and Metadata

The system must support arbitrary binary objects. Each object must have a unique Object ID and maintain, at minimum:

• Object name and size

• Content type

• Creation time and last modified time

• Checksum

• Number of chunks

• Chunk information and storage locations

The system must also support custom user-defined metadata as key-value pairs.

4. Chunking and Replication

Objects must be divided into 1 MB chunks. The final chunk may be smaller than 1 MB. Each chunk must have a unique identifier and its ordering within the object must be preserved.

Each chunk must have two copies during normal operation, with the copies stored on different workers. Chunks should be distributed across the available workers. The system should detect when the required replication factor is no longer maintained and restore it when possible.

5. Client REST API

The following APIs define the required client-facing interface. Structured data should use JSON. Binary object data should be transferred using the appropriate HTTP content type.

Method	Endpoint	Purpose	Success
`POST`	`/api/v1/objects`	`Upload an object`	`201`
`GET`	`/api/v1/objects/{object_id}`	`Download an object`	`200`

Method	Endpoint	Purpose	Success
GET	/api/v1/objects/{object_id}/metadata	Retrieve metadata	200
PATCH	/api/v1/objects/{object_id}/metadata	Update custom metadata	200
DELETE	/api/v1/objects/{object_id}	Delete an object	204
GET	/api/v1/objects	List stored objects	200

5.1 Upload Object

POST /api/v1/objects — Use **multipart/form-data**. Required field: **file** (binary object data). Optional field: **metadata** (JSON containing custom key-value metadata). The system must assign a unique Object ID and return information about the newly stored object.

5.2 Download Object

GET /api/v1/objects/{object_id} — Returns the complete original object as binary data with an appropriate Content-Type. Return **404 Not Found** if the object does not exist. The client should not need to know the worker locations of the object's chunks.

5.3 Get Object Metadata

GET /api/v1/objects/{object_id}/metadata — Returns the metadata of the specified object without downloading its data. The response should include system-maintained metadata and custom metadata.

5.4 Update Object Metadata

PATCH /api/v1/objects/{object_id}/metadata — Accepts a JSON body containing custom metadata to be added or updated. Updating metadata must not require the object data to be uploaded again. Return **404 Not Found** if the object does not exist.

5.5 Delete Object

DELETE /api/v1/objects/{object_id} — Deletes the specified object and its associated stored data. Return **204 No Content** on successful deletion and **404 Not Found** if the object does not exist.

5.6 List Objects

GET /api/v1/objects — Returns the objects currently stored in the system. At minimum, each entry should contain the Object ID, object name, size, and content type. Pagination may be supported for large numbers of objects.

5.7 Error Handling

Status	Usage
200	Successful request
201	Object created
204	Successful deletion
400	Invalid request
404	Object not found
409	Conflict
500	Internal server error
503	Required service/storage unavailable

Error responses should provide a meaningful JSON message.

6. Internal Worker APIs

The following APIs define the basic communication interface between the Controller and Worker nodes. The request and response formats may be extended as required by the implementation.

Method	Endpoint	Purpose
POST	/internal/workers/register	Register a worker with the Controller

Method	Endpoint	Purpose
POST	/internal/workers/heartbeat	Report worker availability
PUT	/internal/chunks/{chunk_id}	Store a chunk on a worker
GET	/internal/chunks/{chunk_id}	Retrieve a chunk from a worker
DELETE	/internal/chunks/{chunk_id}	Delete a chunk from a worker
GET	/internal/workers/{worker_id}/chunks	Retrieve information about chunks stored by a worker

7. Worker Failures

The system must handle storage-worker failures and detect when a worker becomes unavailable. It should continue serving objects when sufficient copies of the required data remain available and should detect when the required replication factor is no longer maintained.

Worker failure should be demonstrated by stopping a Docker container and subsequently restarting it:

```
docker stop worker2
docker start worker2
```

The demonstration should show the system's behavior before, during, and after the failure, including whether the required replication factor is maintained.

8. Data Integrity

The system should provide a mechanism to detect corruption of stored data. It should avoid returning corrupted data when a valid copy is available and handle restoration of corrupted data when possible.

The demonstration should include intentional corruption of stored data.

9. Deployment

• 1 Controller

• 4 Workers

• 1 or more Clients

Each worker must use persistent storage. The project should include instructions for building, starting, and interacting with the system. Docker Compose may be used to simplify deployment. Kubernetes should not be used for worker orchestration.

10. Required Demonstration

• **Object Storage:** Upload and retrieve an object; verify that the retrieved object matches the original.

• **Chunking and Distribution:** Upload an object larger than 1 MB; demonstrate chunking and distribution across workers.

• **Replication:** Demonstrate that each chunk has two copies and show where the copies are stored.

• **Metadata:** Upload custom metadata; retrieve and modify metadata without re-uploading the object.

• **Worker Failure:** Stop a worker container; demonstrate failure handling and object retrieval when another copy is available; restart the worker and demonstrate whether the required replication factor is maintained.

• **Data Integrity:** Intentionally corrupt stored data and demonstrate detection and handling of the corruption.